

Programming 1

Lab manual to accompany COMP100.

01101110
01100001
01110010
01100101
01101110
01100100
01110010
01100001



01110000
01100101
01110010
01110011
01101000
01100001
01100100

Narendra K. Pershad

Software Group, SIT Dept.

September 2025

ASENT³ >

Centennial College

TABLE OF CONTENTS

0 – Introduction	12
1.a – What is Programming?.....	12
1.a.1 – Programming Paradigms	12
1.b – Python	12
1.b.1 – What can be done with Python?	12
1.b.2 – What Cannot Be Done (or Is Not Ideal) with Python?.....	13
1.b.3 – Why use Python?.....	13
1.b.4 – Why use another language?	14
1 – Getting Started.....	15
1.a – Writing/Running Code	15
1.a.1 – Python REPL.....	15
1.a.2 – Running Python code from the command prompt	16
1.a.3 – Running Python code interactively from the command prompt.....	16
1.a.4 – Running Python as an Executable (Unix/Linux/macOS)	17
1.a.5 – Notebooks	17
1.b – Sending output to the screen: using the print() function	18
1.c – Getting input from the keyboard: using the input() function	19
1.d – Comments.....	19
1.e – Variables	19
1.e.1 – Rules for Naming Variable	20
1.e.2 – Naming Conventions (for this course).....	20
1.f – Some useful built-in functions	21
1.f.1 – id()	21
1.f.2 – type()	22
1.f.3 – dir()	22
1.f.4 – help()	23
1.f.5 – eval().....	24
1.g – Summary	24
2 – Datatypes.....	26
2.a – None	27
2.b – Numerics.....	27
2.b.1 – Integers.....	27
2.b.2 – Floating Point Numbers.....	30
2.b.3 – Complex Numbers	31
2.b.4 – Fraction (informational)	31
2.b.5 – Decimal (informational)	31

2.c – Sequences:	31
2.c.1 – Strings	31
2.c.2 – Tuple	32
2.c.3 – Range	33
2.c.4 – List	33
2.d – Mappings:	33
2.d.1 – Dictionary	33
2.e – Set types	34
2.e.1 – Set	34
2.f – Binary types	34
2.g – Enum	35
2.h – Checking Data Types	35
2.i – Converting Data Types	36
2.i.1 – Implicit	36
2.i.2 – Explicit	36
2.j – Operators	37
2.j.1 – Assignment Operators	38
2.j.1 – Arithmetic Operators	38
2.j.2 – Relational (Comparison) Operators	39
2.j.3 – Logical Operators	40
2.j.4 – Identity Operators	41
2.j.5 – Membership Operators	42
2.j.6 – Bitwise Operators	42
2.j.7 – Operator precedence	43
2.k – Summary	44
3 – Control Structures: Sequential and Conditionals	46
3.a – Sequential	46
3.b – Conditionals	47
3.b.1 – if	48
3.b.2 – else	48
3.b.3 – elif	48
3.b.4 – and	49
3.b.5 – or	49
3.b.6 – not	49
3.b.7 – Ternary statements	49
3.c – Comparison Operators	50
3.c.1 – Greater than (>) operator	51
3.c.2 – Less than (<) operator	52
3.c.3 – Greater than or equal (>=) operator	53

3.c.4 – Less than or equal (<=) operator	53
3.c.5 – Equality (==) operator	54
3.c.6 – Not equals (!=) operator	54
3.d – Logic Operators	55
3.d.1 – Logical and operator.....	55
3.d.2 – Logical or operator	56
3.d.3 – Logical not operator	56
3.d.3.1 – Bit fiddling	57
3.d.4 – Best Practice	59
3.e – Match statement	59
3.f – Summary	61
4 – Repetition.....	62
4.a – While statements	62
4.b.1 – For loop.....	63
4.b.1.1 – Iterating Over an Iterable	63
4.b.1.2 – Using the range object	64
4.b.1.3 – Using the Enumerate Function.....	65
4.b.1.4 – Nested Loop	67
4.c – Loop control Keywords.....	67
4.c.1 – break	67
4.c.2 – continue	68
4.c.3 – pass	68
4.d – Infinite Loops	68
4.d – Summary.....	69
5 – Functions.....	70
5.a – User-Defined Functions.....	70
5.a.1 – Creating Functions.....	70
5.a.2 – Calling Functions.....	71
5.a.3 – Functions with Inputs	71
5.a.4 – Functions with return value.....	71
5.a.5 – Variable Scope	72
5.a.6 – Keyword Arguments	72
5.a.7 – Default Arguments	72
5.a.8 – Annotation.....	73
5.a.9 – Recursive Functions.....	74
5.a.10 – Inner Functions	75
5.a.11 – Higher-order Functions.....	76
5.a.12 – Closures	77
5.a.13 – Lambda Closures.....	78

5.b – Documentation	79
5.b.1 – Code Explanation.....	79
5.b.2 – Disabling Code	79
5.b.3 – To-Do Notes.....	79
5.b.4 – Code Maintenance	80
5.b.5 – Code Review	80
5.b.6 – Debugging Information	80
5.b.7 – Best Practices for Writing Comments	80
5.c – Built-in Functions	81
Randomisation	84
range	84
round.....	85
abs	85
6 – Sequences	86
Immutables	86
Mutables	87
6.a – String	87
6.a.1 – String Creation Methods	88
6.a.1 – String Concatenation	88
6.a.2 – Escaping a String	89
6.a.3 – Raw strings	89
6.a.4 – Combining F-Strings and raw strings	90
6.b – List	90
6.b.1 – Creating Lists	90
6.b.2 – Adding Lists Together	90
6.b.3 – Adding an Item to a List.....	91
The in operator.....	91
List Indexing.....	91
List Slicing	91
6.c – Common Sequence Operations	92
6.b.1 – min(), max() sum()	92
6.b.1 – Length.....	92
6.b.1 – Membership test	92
6.b.1 – Concatenation	92
6.b.1 – Repetition.....	92
6.b.1 – Iteration.....	92
6.b.1 – Indexing	92
6.b.1 – Slicing.....	92
8 – Sets & Mappings.....	93
Set.....	93
Creating sets	93

add(x)	94
clear()	94
copy().....	94
difference(x)	94
difference_update(iter).....	95
discard(x)	95
intersection()	95
intersection_update()	95
isdisjoint(x)	95
issubset()	96
issuperset().....	96
pop()	96
remove()	96
symmetric_difference().....	97
symmetric_difference_update()	97
union()	97
update().....	97
Dictionary.....	98
9 – File I/O & Errors.....	99
9.a – Working with files.....	99
9.a.1 – Opening a file.....	99
9.a.2 – Writing to a file	99
9.a.3 – Reading from a file	99
9.a.3 – A more elegant way of reading/writing a file	100
Operation System File Management.....	100
Checking for the existence of a file	100
Deleting a file	100
Deleting a folder	101
Creating a folder	101
9.c – Errors	101
9.c.1 – Syntax Error	101
9.c.1 – Name Error	101
9.c.2 – Type Error	102
9.c.3 – Value Error	102
9.c.4 – Index Error	102
9.c.5 – Key Error	102
9.c.6 – FileNotFoundError	102
9.c.7 – Zero Division Error	103
9.c.1 – Error handling	103
10 – Packages/Modules	104
10.a – Creating your own Module.....	104
10.b – Creating your own Package	105

10.c – Importing/Loading a Module	105
10.c.1 – Importing specific items from a module.....	105
10.c.2 – Importing Everything	106
10.c.3 – Aliasing.....	106
10.d – Using the Module	106
10.e – Removing a Module	107
11 – Object Oriented Programming	108
Classes and Objects.....	108
Defining a Python Class	108
Instantiating the Class.....	108
The <code>__init__</code> method.....	108
Instance Variables.....	109
Instance Methods	110
Class Methods	110
Class Variables.....	110
Class Inheritance.....	111
Appendices	112
a – Programming Paradigms.....	112
a.1 – Imperative Programming.....	112
a.2 – Procedural Programming.....	112
a.3 – Object-Oriented Programming (OOP)	113
a.4 – Functional Programming	113
a.5 – Declarative Programming	113
Why are Programming Paradigms Important?	114
a.6 – The Best Paradigm	114
b – What can be done with Python?	114
b.1 – Machine Learning and Artificial Intelligence	114
c – Why use Python?	115
d – Why you might consider another language?	115
Docstring	115
3.a – None	116
Debugging	116
Types of Errors.....	117
Syntax Errors:	117

Runtime Errors:	117
Logical Errors:.....	118
Using print statement	118
Using logging	119
Using Python Debugger (pdb).....	119
Using IDE Debugger (VS Code)	119
Setting breakpoints	119
Stepping through code	119
Examining Variables	119
<i>Iterators.....</i>	<i>119</i>
a – The Iterator Protocol	119
b – Using Built-in Iterator	119
c – Practical Benefits	120
d – Iterables vs Iterators.....	120
e – Creating Custom Iterators.....	121
f – When not to Prefer Iterators	121
<i>Generators.....</i>	<i>122</i>
Generator Expressions	122
<i>Using PIP</i>	<i>123</i>
Default Installation	123
Install a specific version	123
Install Extra	123
Install from Source.....	123
Install from Version Control	123
Install from other Package Indexes	123
Upgrading packages	123
Using a requirements file	123
Freezing dependencies.....	123
<i>CoPilot</i>	<i>123</i>
a – Installing Copilot	123
b – Enabling Copilot	124

c – Disabling Copilot	124
d – Tips of using Copilot	124
i – Write clear comments	124
ii – Use descriptiveWrite clear comments.....	124
Virtual Environments	125
a – Creating Virtual Environments	125
b – Activating the Virtual Environment	126
b.1 – On Windows	126
b.2 – On macOS/Linux.....	127
c – Installing Packages	127
c.1 – Batch Installation	128
d – Deactivating the Virtual Environment	129
e – Deleting a Virtual Environment	129
Regular Expression	130
match().....	130
search()	130
findall().....	131
finditer().....	131
sub().....	131
Exception Handling	132
Testing	132
Types of Testing	132
Unit Testing	132
Integration Testing	132
Functional Testing	132
Regression Testing.....	132
End-to-End Testing.....	132
Tools	133
unittest	133
pytest.....	133
doctest.....	133
Basic Testing Workflow	133
Modules and Packages	133
a – Module	133
a.1 – Importing the module.....	134

a.2 – Import using alias	135
a.3 – Import only selected objects	135
a.4 – Import everything.....	135
b – Package.....	136
c – Library.....	136
d – Framework.....	136
<i>Extending Python.....</i>	<i>136</i>
Install the Python.Net package.....	136
Write the C# code	137
Compile to a DLL.....	137
Load the library and invoke the Greet() method	137
Other languages.....	138
When not to extend python	138
<i>Keywords.....</i>	<i>138</i>
<i>0 – Glossary</i>	<i>140</i>
Assignment.....	140
Indexable	140
Iterable	140
Sequences	140
Collections	140
Mappings.....	140
Method.....	141
Function.....	141
Overloading	141
Inheritance	141
Type Annotation.....	141
Docstring	141
Comments	141
Repl	141
IDE	141
Debugging	141
Operator	141
Keyword	141
Module	141
Import.....	141
Algorithm.....	141
Boolean (bool).....	142
Collection.....	142
Comment.....	142

# This is a comment	142
Data Type	142
Debugging	142
Dictionary (dict)	142
Docstring	143
Function	143
Method	143
Indentation	143
Inheritance	143
Iterable	144
Keyword	144
List	144
Loop	144
Module	144
Import	144
Mutable	145
Operator	145
Sequence	145
Tuple	145
Type Annotation	146
Variable	146
2.j.7 – Operator precedence	147

0 – INTRODUCTION

1.a – What is Programming?

Programming is the art and science of giving a computer a precise, logical set of instructions to solve a problem or perform a task. It involves analyzing a problem, breaking it down into smaller more manageable steps, and expressing those steps in a way that a computer can understand and execute.

Programming relies on fundamental concepts such as **variables**, **data types** and **controls structures** to build functional software systems such as websites, mobile applications, games, operating systems embedded systems and AI-driven apps.

1.a.1 – Programming Paradigms

A programming paradigm is a set of principles or a style that guides how programs are structured and how problems are solved. It influences which tools and techniques are preferred and how they are combined to produce reliable and maintainable software.

Common paradigms include:

- Imperative
- Declarative
- Functional
- Object-Oriented

These paradigms are discussed in more detail in the appendix of this document.

1.b – Python

Python is a high-level, interpreted, and general-purpose programming language created by Guido van Rossum and first released in 1991. It emphasizes code readability and simplicity allowing programmers to express ideas using fewer lines of code compare to languages such as C++ or Java.

1.b.1 – What can be done with Python?

It excels in areas where developer productivity, readability, and the ability to leverage powerful libraries are more important than raw execution speed or low-level control. It is able to work seamlessly with external libraries.

- **Data Science, Analytics and Visualization:** This is one of Python's biggest strongholds. Libraries such as Pandas, NumPy, Matplotlib and Seaborn makes it a powerhouse for working with data.
- **Machine Learning and Artificial Intelligence:** Libraries like scikit-learn, TensorFlow, PyTorch and Keras are high-performant and can take advantage of specialize hardware designed for AI.
- **Back-End Web Development:** there are excellent libraries such as Django, Flask and FastAPI.
- **Automation, Scription and DevOps**
- **Scientific and Numeric Computing:** Access to high performance libraries like NumPy, SciPy.
- **Web Scraping**
- **Software Prototyping:** It can quickly validate an idea or building a Minimum Viable Product (MVP)
- **Education:** It is the most popular introductory teaching language in universities, colleges and coding bootcamps worldwide

1.b.2 – What Cannot Be Done (or Is Not Ideal) with Python?

Although Python is a very versatile, it is not always the best tools for every tasks. Its limitation typically falls into the areas of performance, system-level programming and constrained environment.

Examples includes:

- **System-level Programming** such as operating systems or device drivers.
- **Real-time or Ultra-High-performance Applications.**
- **Memory-Intensive Tasks** in constrained environments.
- **Browser/Client-Side Web Development.**
- **Statically-Typed and Secure-By-Design Systems.**

1.b.3 – Why use Python?

Python is a powerful, accessible, and extremely popular language that acts as a Swiss Army knife for programmers. It is suitable for beginners and professionals alike, with a massive ecosystem of libraries and a strong global community. Its emphasis on clear and readable code makes it an excellent first language as well as a professional tool used by organizations such as Google, Netflix, Instagram, and NASA.

1.b.4 – Why use another language?

- **CPU Intensive Tasks:** Python being an interpreted and high-level language is slower than compiled languages like C++.
- **Mobile Applications:** Swift, Kotlin, Java Flutter or React Native are better suited.
- **Compile-Time Error Detection:** Python's dynamic typing can allow some errors to surface only at runtime.
- **In-Browser Applications:** JavaScript or TypeScript are required.

1 – GETTING STARTED

This chapter introduces the fundamentals needed to begin working with Python. You will learn how to write and run Python programs using different tools and environments, ranging from interactive sessions to full program execution. Along the way, you'll explore how Python displays output, accepts input from the keyboard, and uses variables to store data.

You'll also be introduced to comments, naming conventions, and several built-in functions that help you understand and explore Python code. By the end of this chapter, you will have the foundation needed to comfortably experiment with Python, read simple programs, and prepare for more advanced topics later in the course.

1.a – Writing/Running Code

There are several common ways to run Python code. Each useful in different situations:

- Interactive notebooks (VS Code, Jupyter, Google Colab and PyCharm)
- The Python REPL
- Running a script from the command line
- Executing a script as an executable file

Fun fact

A Pythonista is someone who enjoys using Python – whether they're a beginner, expert, or somewhere between.

1.a.1 – Python REPL

The Python REPL is one of the simplest and most flexible ways to run Python code. It lets you write one line at a time and get immediate feedback, which makes it popular with both beginners and veteran *Pythonistas*.

To use the REPL, you start the python interpreter. This opens a command prompt similar to your operating system shells. This is a **Read-Evaluate-Print Loop**.

- Python *reads* your input
- *evaluates* it
- *prints* the result back to the screen

You can use this technique to incrementally build substantial python applications or parts of applications. You are able to examine each object at your leisure. You may also debug problematic code this way.

The downside is that it is not convenient for larger/longer block of code or distributing code to end users or when typing is challenging.

Below is a sample session in the python **repl** in a windows 10 environment:

```
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\progs\new100\labs> python
Python 3.10.7 (tags/v3.10.7:6cc6b13, Sep  5 2022, 14:08:36) [MSC v.1933 64
bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> 2 + 6
8
>>> 8 / 3
2.6666666666666665
>>>
>>> hi = 'hi '
>>> hi = hi * 4
>>> hi
'hi hi hi hi '
```

1.a.2 – Running Python code from the command prompt

Once your program is complete and behaving correctly (ideally 95-100% according to specifications), you typically run it from the command line.

```
«path_to_python» «name_of_code_file»
```

```
>>> python lec33_docstring.py
```

1.a.3 – Running Python code interactively from the command prompt

The above technique, terminates the python session after the python file is processed. This means you can't inspect variables or object afterwards.

If you use the `-i` switch, Python keeps the session alive. This allows you to examine variables, functions and objects created with the program.


```
«path_to_python» «name_of_code_file»
```

```
>>> python -i lec33_docstring.py
```

This is especially useful for debugging and exploration.

1.a.4 – Running Python as an Executable (Unix/Linux/macOS)

In this method of running python code there are two steps involved, changing the permission of the file and then inserting the path to the python interpreter in the first line of the code file.

```
chmod 755 «name_of_code_file»
```

or

```
chmod a+x «name_of_code_file»
```

And then modifying the code file, insert the following at the top of the file:

```
#!/«path_name_of_python_interpreter»
```

This is probably the best and the most professional way of running or distributing your code to everyone from the novice user to the seasoned developer.

This is one of the most professional ways to distribute Python programs to users of all experience levels.

Interesting fact

This approach works only on operating systems that support file permissions (Linux, Unix, macOS). On Windows, you can achieve a similar result using batch files or PowerShell scripts.

1.a.5 – Notebooks

Notebooks are quickly becoming one of the most popular ways to write, run and demonstrate Python code.

A notebook is made up of three types of cells:

- **Markdown cells** for text and explanations

- **Code cells** containing Python code
- **Output cells** that show the results of running code

Notebooks feels like browsing a live textbook: you can read the theory and see the code, run it, and immediately view the results – all in one place. You can also modify the code and re-run to observe changes.

Many online platforms provide pre-configures environs with required libraries already installed. This is especially helpful for beginners or when working with complex setups like machine learning or image processing.

Popular notebook platform includes:

- Jupyter
- Binder
- Kaggle
- Google
- Microsoft
- CoCalc
- Datalore.

1.b – Sending output to the screen: using the `print()` function

The `print()` function displays output on the console. While it's simple to use, it also supports more advanced options that we will explore later in the semester.

```
>>> print(4)                                #displays the integer literal 4
4

>>> print(3 + 0.141592)                     #displays the float literal
3.141592

>>> print('Hello World')                   #displays the string literal
Hello World

>>> print(name)                             #display the contents of variable name
Narendra Pershad
```

1.c – Getting input from the keyboard: using the input() function

The `input()` function displays a prompt and waits for the user to type something. The results is always returned as a string, so it's usually stored in a variable for later use.

```
>>> name = input('What's your name? ')
What's your name? Narendra          #user response is shown in bold face

>>> print(name)
Narendra
```

In the next chapter, we will learn how to convert this text input to numbers and other data types.

1.d – Comments

Comments are lines of text that are ignored by the interpreter. They are used to improve the readability of the code for coders themselves or other developers. For single-line comments, use the “#” symbol, for multi-line comments use either triple single or double quotes.

Fun fact

Comments are not written in the repl.

```
#this is a single-line comment

'''
This is a multi-line comment.
It can span more than
one line
'''
```

Because Python code is generally easy to read, avoid unnecessary comment that restates the obvious. Use comments sparingly to explain why something is done, and not what the code already makes clear

1.e – Variables

At the infancy of computing, when computer scientist wants to use a piece of memory, they have to use a memory address to refer to that location, this is both tedious and error-prone. Shortly afterwards, language designers allow programmers

to assign a name to a memory location. So instead of referring to numeric memory addresses the programmer uses descriptive names to manipulate values.

A variable gives a name to a piece of data (or location of that data). Like a box with a label, you may access what's inside the box if you know the name of the box.

```
name = 'Narendra'
age = 12
```

Once a variable is created in the REPL, it remains until the current session is destroyed. You can remove a variable using the `del` keyword.

1.e.1 – Rules for Naming Variable

- Variable must start with a letter or underscore
- Can not contain spaces
- Cannot be a python keyword
- Cannot contain include symbols like '!', '@', '\$', '#', or '%'
- Avoid using name of built-in function 'list', 'str', 'sum', 'int', or 'len'.
- Avoid starting name with double underscores.

```
>>> student_1 = 'Justin Trudeau'
>>> list = [3]                #not legal
>>> first_name = 'Narendra'    #not legal
```

1.e.2 – Naming Conventions (for this course)

- Use descriptive names
- Avoid single letter unless they are meaningful in the context.
- Use **snake-casing** for variables
- Use **all caps** for constants.
- Avoid starting variables with single or double underscores.

Fun fact

C# uses camelcase for variables and pascalcase for methods.

```
>>> first_name = 'Narendra'
```

```
>>> GST = 0.14                #suggest to the programmer GST is a
constant
>>> __first_name = 'Narendra'  #legal but not recommended
>>> a = 3                      #legal but not descriptive
```

1.f – Some useful built-in functions

Python includes many built-in functions that are available without any setup. These functions solve common problems and save time.

FYI

We will cover function formally later in chapter 5.

In addition to `print()` and `input()` here are some frequently used ones.

1.f.1 – `id()`

The `id()` function returns a unique identifier for an object. In CPython, this usually corresponds to the object's memory address.

The following code illustrates that `int` are immutable and `list` are mutable.

```
>>> a = 3
>>> id(a)
1275421851952                #address of the int a

>>> a = a + 7
>>> id(a)
1275421852176                #address of the int a (different)
                             # ask your instructor to explain this

>>> b = []
>>> id(b)
1275429377408                #address of the list b

>>> b.append(3)               # adds an item to the list
>>> b
[3]

>>> id(b)
1275429377408                #address of the list b (identical)
```

ask your instructor to explain this

1.f.2 – type()

The `type()` function tells you the data type of an object:

FYI

We will cover datatypes in the next chapter.

```
>>> type(3)
<class 'int'>

>>> type(3.1415)
<class 'float'>

>>> type('Narendra')
<class 'str'>

>>> type(True)
<class 'bool'>

>>> type([])
<class 'list'>

>>> type((4,))
<class 'tuple'>

>>> type({'a'})
<class 'set'>
```

1.f.3 – dir()

The `dir()` function shows all the objects that are currently in scope. The first usage shows the objects in the global scope and the second usage show the objects in the `__builtins__` namespace.

FYI

A more powerful way to explore Python's built-in documentation system is to use the command:

`python -m pydoc -b`

```
>>> dir()
['__annotations__', '__builtins__', '__doc__', '__loader__', '__name__',
 '__package__', '__spec__', 'a', 'b']
```

```
>>> dir(__builtins__)
['ArithmeticError', 'AssertionError', 'AttributeError', 'BaseException',
...
'setattr', 'slice', 'sorted', 'staticmethod', 'str', 'sum', 'super',
'tuple', 'type', 'vars', 'zip']
```

This function is very useful for introspection. It allows you to list the attributes and methods of an object especially in the repl.

1.f.4 – help()

The `help()` function launches Python built-in help system. Invoking the `help()` function with `str` and `str.split` arguments.

```
>>> help(str)
Help on class str in module builtins:

class str(object)
|   str(object='') -> str
|   str(bytes_or_buffer[, encoding[, errors]]) -> str
|
|   Create a new string object from the given object. If encoding or
|   errors is specified, then the object must expose a data buffer
```

```
>>> help(str.split)
Help on method_descriptor:

split(self, /, sep=None, maxsplit=-1)
```

Return a list of the substrings in the string, using `sep` as the separator string.

`sep`

The separator used to split the string.

It provides interactive access to Python's documentation. It is useful for quickly getting information about modules, classes, functions, keywords and other Python objects.

1.f.5 – `eval()`

The `eval()` function evaluates strings as Python code and returns the result.

While it can be useful in controlled situations, **never use `eval()` on untrusted input**. It can execute malicious code and should be avoided in production programs

```
>>> eval('2 + 3')
5

>>> list_string = "[1, 2, 3, 'hello', 4.5]"
>>> my_list = eval(list_string)

>>> print(my_list
[1, 2, 3, 'hello', 4.5]

>>> print(type(my_list))
```

1.g – Summary

In this chapter, we explored the basics of getting started with Python and the different ways Python code can be written and executed. We learned that Python programs can be run using the REPL, from the command line, as executable scripts on Unix-based systems, or through interactive notebooks such as Jupyter and Google Colab. Each approach serves a different purpose, from quick experimentation to professional code distribution.

We introduced the Python REPL as an interactive environment that allows immediate feedback, making it ideal for learning, testing, and debugging small pieces of code. We also saw how Python scripts can be executed from the command prompt, including the use of the `-i` switch to keep the session alive for further inspection.

The chapter covered basic input and output using the `print()` and `input()` functions, emphasizing that user input is always received as text. We discussed comments and their role in improving code readability, along with best practices for using them effectively.

We examined variables as named references to data, reviewed the rules and conventions for naming them, and introduced the idea of constants. Finally, we explored several useful built-in functions such as `id()`, `type()`, `dir()`, and `help()`, which help with understanding objects, data types, and Python's built-in features.

By the end of this chapter, you should be comfortable running Python code in multiple environments, displaying output, accepting user input, working with variables, and using basic built-in tools to explore and understand Python programs.